	훈련교사	담 당	팀/과 장
확인			

프로젝트 안내

프로젝트명	YOLOv8 기반 도로 위 차량·보행자 실시간 인식 및 교통량 카운팅 (TrafficLens)		
프로젝트 기간	**2026.10.13 (화) ~ 2026.10.31 (토)**		
팀명	로드비전	**작업(완료) 예정일자**	2026/10/31 (토)
팀원	장창근(팀장), 반광수, 지명준, 장혜원		

작업내역 or 개요

<프로젝트 개요>

도로 영상에서 YOLOv8 로 차량 4 종(승용차·버스·트럭·오토바이)과 보행자를 탐지하고, ByteTrack 으로 추적해 화면에 그은 기준선을 지나간 수를 방향별로 세는 교통량 인식 시스템을 만들었습니다. 별도 학습 없이 COCO 사전학습 가중치를 쓰고, 높은 곳에서 찍은 영상의 작은 보행자(약 20px)를 잡기 위해 YOLOv8s · 입력 1280 을 적용했습니다. 신호등이 보행자로 잡히는 오탐지와 누적 개수(total) 부풀림은 정지 물체 필터 · 클래스 다수결 · 짧은 추적 제거 · 중복 박스 제거로 줄였고(차량 total 753 → 485, 교통량 line 151 대 유지), 결과를 소개 사이트(https://trafficlens.maxfinup.com/)로 공개했습니다.

<프로젝트 일정&기간>

회차	목표	기간
1 차	사전 기획, 개발 환경 구축, 기본 시스템 구현 (탐지·추적·기준선 카운팅)	26.10.13 ~ 26.10.14 (1 주차)
2 차	오탐지·누적 개수 부풀림 개선 (개선 기능 4 종)	26.10.13 ~ 26.10.14 (1 주차)
3 차	성능 측정 (mAP·FPS, 입력 해상도 640 / 1280 비교)	26.10.14 ~ 26.10.15 (1 주차)
4 차	발표 (발표자료 20 장, 대본·예상 질문)	26.10.13 ~ 26.10.16 (1 주차)
5 차	소개 사이트·보고서·Colab 노트북·코드 저장소 정리	26.10.13 ~ 26.10.16 (1 주차)
6 차	정답(직접 센 수) 비교 평가 및 최종 마무리	26.10.19 ~ 26.10.31 (2~3 주차)

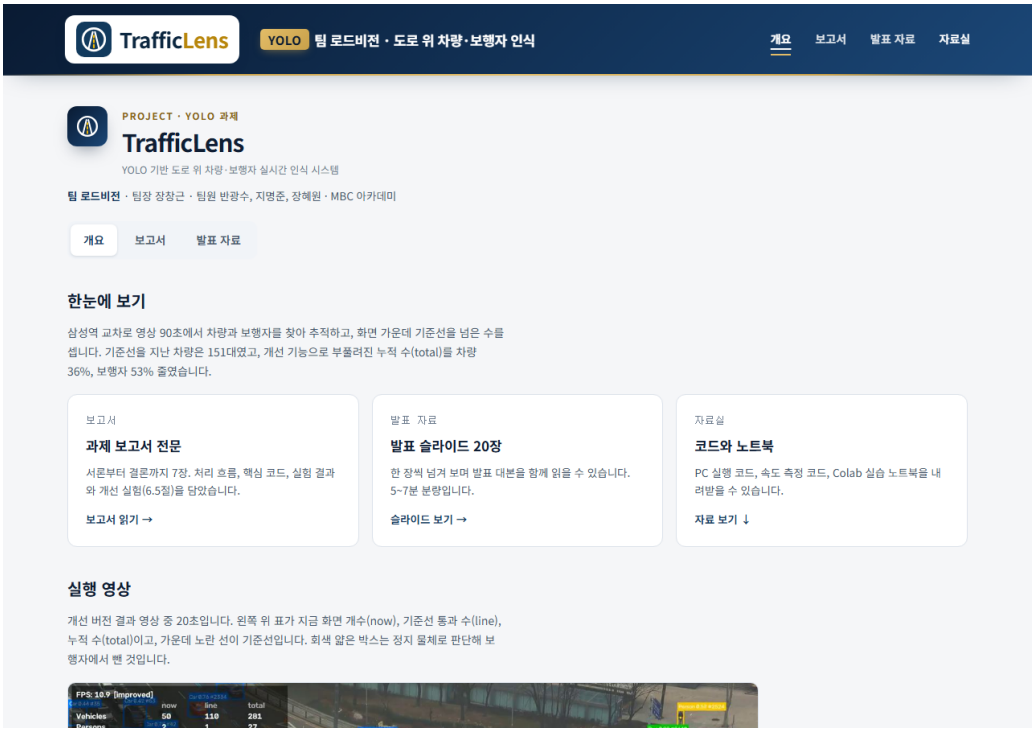
<팀원 역할 분담>

팀원	역할
장창근 (팀장)	프로젝트 기획·일정 관리, 발표 (2026.10.13)
반광수	
지명준	
장혜원	

시연 사이트 or 작업일정 및 역할

시연 사이트 https://trafficlens.maxfinup.com/

- **/** 프로젝트 개요 (한눈에 보기 · 실행 영상 · 처리 흐름 · 결과 · 로고 시안 7 종)
- **/report/** 보고서 전문 — 서론부터 실험 결과 · 결론, 부록(발표 자료)까지
- **/slides/** 발표 슬라이드 20 장과 장별 발표 대본
- **/#tl-files** 자료실 — Colab 노트북, traffic_detect.py, evaluate.py, 개선 버전 미리보기 영상 내려받기



trafficlens.maxfinup.com 첫 화면 — 팀 소개, 한눈에 보기(기준선 통과 151 대, total 차량 −36% · 보행자 −53%)

프로젝트 일지(1 차)

작업내역

<프로젝트 목표>

회차	목표	기간
1 차	사전 기획, 개발 환경 구축, 기본 시스템 구현	26.10.13 ~ 26.10.14

<요약>

주제 선정(도로 교통량 자동 집계)과 목표 설정, 테스트 영상(삼성역 교차로) 확보.

YOLOv8s + ByteTrack 으로 탐지 → 추적 → 기준선 카운팅을 하는 traffic_detect.py 기본 버전 구현.

90 초 영상에서 기준선 통과 차량 151 대를 집계했지만, 신호등 오탐지와 total 부풀림을 발견해 2 차에서 개선.

세부내역

[사전 기획]

도로 위 차량과 보행자를 사람이 직접 감시하면 인력과 시간이 많이 들고 놓치는 상황이 생깁니다. YOLO 는 한 번에 위치와 종류를 예측해 속도가 빨라, CCTV 영상을 실시간으로 처리해야 하는 교통 분야에 적합하다고 판단했습니다.

프로젝트 목표는 아래와 같이 정했습니다.

[1] 사전 학습된 YOLOv8 로 차량(승용차 · 버스 · 트럭 · 오토바이)과 보행자를 탐지

[2] 탐지한 객체를 종류별로 세어 화면에 실시간 표시하고, 기준선 통과 수로 방향별 교통량 집계

[3] 모델 크기별 정확도(mAP)와 처리 속도(FPS)를 비교해 실시간 적용 가능성 확인

활용 분야는 스마트 교차로의 신호 제어 · 교통량 조사, 어린이 보호구역 보행자 감지, 자율주행 차량의 주변 인식으로 잡았습니다.

[테스트 영상과 탐지 클래스]

서울 삼성역 교차로를 높은 곳에서 찍은 YouTube 영상(Spotting Korea,「삼성역 교차로 자동차 영상 4K Spotting Seoul」)을 1080p 로 받아, 0:30 ~ 2:00 구간 90 초(2,698 프레임)를 road.mp4 로 잘라 썼습니다. 학습 없이 COCO(80 개 클래스) 사전학습 가중치에서 교통 관련 5 개 클래스만 골랐습니다.

COCO ID	클래스	화면 표시
0	person	보행자
2	car	승용차

3	motorcycle	오토바이
5	bus	버스
7	truck	트럭

[개발 환경]

구분	내용
로컬	Windows 10, Intel Core i5-6600, 메모리 16GB, GeForce GTX 1060 3GB
클라우드	Google Colab (실습 노트북 TrafficLens_Colab.ipynb)
언어·라이브러리	Python, Ultralytics 8.4 (YOLOv8), PyTorch 2.14 (CUDA 12.6), OpenCV, lap(ByteTrack)
산출물	traffic_detect.py, evaluate.py, compare_resolution.py, Colab 노트북, 결과 영상

[기본 시스템 구현 — traffic_detect.py]

프레임마다 탐지 → 추적 → 카운팅 → 화면 표시 → 저장을 반복합니다.

[1] 클래스 필터: classes=[0, 2, 3, 5, 7] 로 보행자와 차량 4 종만 탐지

[2] 작은 보행자 대응: YOLOv8s · 입력 1280, 보행자 신뢰도 기준만 0.25 (차량은 0.4)

[3] 객체 추적: model.track(persist=True, tracker="bytetrack.yaml") 이 프레임이 바뀌어도 같은 ID 를 유지

[4] 기준선 카운팅(line): 객체 중심의 이동 경로가 기준선을 가로지르면 방향별로 한 번만 센다

[5] 화면 표시: 클래스별 색상 박스, ID · 신뢰도, 노란 기준선(통과 순간 초록), now · line · total 집계표, FPS

화면 표시	뜻
now	지금 화면에 보이는 개수
line	기준선을 지나간 수(방향별) — 교통량, 평가에 쓰는 값
total	지금까지 나온 고유 추적 ID 수 — 가려짐 · 클래스 혼동으로 부풀기 쉬움

[기본 버전 결과 (90 초 · result.mp4)]

구분	line (기준선 통과)	total (누적 고유 ID)
승용차	146 (방향별 75 / 71)	531
버스	3	125
트럭	2	97
오토바이	0	0
보행자	1	75



기본 버전 결과 화면 (90 초 끝 무렵) — 왼쪽 위 now · line · total 집계표, 가운데 노란 세로선이 기준선

[1] 화면에 동시에 있는 차량 50 ~ 60 대를 대부분 탐지했고, 승용차 line 은 양방향 75 / 71 대로 고르게 집계됐습니다.

[2] 신호등이 보행자로 잡혔습니다. 보행자 신뢰도 기준을 0.25 로 낮춘 대가입니다.

[3] 짐칸 트럭이 버스로 잡히는 등 같은 차의 클래스가 프레임마다 바뀌었고, 가려졌다 나타나면 새 ID 가 붙어 버스 total 이 125 까지 부풀었습니다. 실제 교통량은 line 값이 더 믿을 만합니다.

프로젝트 일지(2 차)

작업내역

<프로젝트 목표>

회차	목표	기간
2 차	오탐지·누적 개수 부풀림 개선 (개선 기능 4 종)	26.10.13 ~ 26.10.14

<요약>

1 차에서 찾은 문제 3 가지에 맞춰 정지 물체 필터 · 클래스 다수결 · 짧은 추적 제거를 넣은 개선 버전 구현.

차량 total 753 → 485(−36%), 보행자 total 75 → 35(−53%), 교통량 line 151 대는 그대로.

서 있는 트럭에 겹친 Bus 박스의 원인(클래스별 NMS)을 찾아 중복 박스 제거 추가 → 버스 total 67 → 29.

세부내역

[문제와 개선 방법]

개선 기능은 traffic_detect.py 에 기본으로 켜져 있고, --baseline 으로 끄면 기본 버전과 비교할 수 있습니다.

문제	개선 방법
신호등이 보행자로 오탐지	정지 물체 필터: 화면을 32px 칸으로 나눠, 20 초 이후 프레임의 15% 이상 '보행자'가 나온 칸의 보행자는 제외. 신호등은 ID 가 계속 바뀌므로 ID 가 아닌 위치로 판단
트럭 · 버스 클래스 혼동	클래스 다수결: 추적 ID 별로 지금까지 가장 많이 나온 클래스를 사용
잠깐 나타난 ID 로 total 부풀림	짧은 추적 제거: 5 프레임보다 짧게 보인 ID 는 total 에서 제외
한 차량에 다른 클래스 박스가 겹침	중복 박스 제거: 박스가 크게 겹치면(--dup-iou 0.8) 오래 추적된 쪽만 남김

[개선 실험 결과 (같은 90 초 영상)]

구분	기본 line	개선 line	기본 total	개선 total
승용차	146	144	531	391
버스	3	3	125	67

트럭	2	4	97	27
차량 합계	151	151	753	485
보행자	1	1	75	35



개선 버전 결과 화면 (90 초 끝 무렵) — 차량 line 151 은 그대로, total 은 753 → 485

[1] 정지 물체 필터: 21 개 ID 가 정지 물체로 판정됐고, 같은 장면에서 신호등 5 개 중 3 개가 걸러졌습니다. 영상 후반에 처음 나타난 2 개는 비율이 기준에 못 미쳐 남았습니다.

[2] 클래스 다수결: 차량 line 합계는 151 로 같고, 승용차 2 대가 트럭으로 다시 분류됐습니다.

[3] 속도: 영상을 저장하지 않으면 기본 16.7 FPS, 개선 16.8 FPS 로 차이가 없었습니다. 1080p 결과 영상을 저장하면 약 11.9 FPS 로 떨어져 영상 저장이 가장 큰 병목이었습니다.

[4] 한계: 신호등 위치의 '보행자' 비율은 18 ~ 45% 였지만, 실제 보행자가 신호를 기다리는 자리도 28% 가 나와 한자리에 오래 서 있는 보행자도 빠질 수 있습니다.

[중복 박스 원인 찾기 — 서 있는 트럭의 Bus 박스]

개선 버전 영상에서 주차된 트럭(#39) 위에 Bus 박스가 계속 겹쳐 보였습니다. YOLO 의 NMS 가 클래스별로 돌아 같은 차에 Truck 과 Bus 박스가 함께 남는 것이 원인이었습니다. 중복 박스 제거(drop_duplicates)를 넣자 버스 total 이 67 → 29 로 줄었고 line 값은 변하지 않았습니다. 발표 시연 영상은 이 버전(result_improved_v2.mp4)입니다.

오른쪽 위의 작은 Person 박스(#2524)는 신호등이 아니라 광장을 걷는 실제 보행자였습니다.

프로젝트 일지(3 차)

작업내역

<프로젝트 목표>

회차	목표	기간
3 차	성능 측정 (mAP·FPS, 입력 해상도 640 / 1280 비교)	26.10.14 ~ 26.10.15

<요약>

evaluate.py 로 COCO val2017(5,000 장)에서 교통 5 개 클래스 mAP 와 GPU 탐지 FPS 를 모델 · 해상도별로 측정.

COCO 에서는 YOLOv8s · 1280(0.438)이 640(0.513)보다 낮아, 1280 을 쓰는 근거를 따로 확인.

compare_resolution.py 로 테스트 영상의 사람 탐지 수 비교 → 640 에서 12 명, 1280 에서 190 명.

세부내역

[측정 방법 — evaluate.py]

COCO val2017 사진 5,000 장과 라벨을 받아 사람 · 승용차 · 오토바이 · 버스 · 트럭 5 개 클래스의 mAP 를 재고, 같은 실행에서 GPU(GTX 1060) 탐지 FPS 도 함께 쟀습니다. coco.yaml 을 그대로 쓰면 학습용 데이터 19GB 까지 내려받으므로 val 데이터만 썼습니다.

[정확도 · 속도 측정 결과 (2026.10.14)]

모델	해상도	mAP50-95	mAP50	GPU FPS
YOLOv8n	640	0.438	0.626	60.5
YOLOv8s	640	0.513	0.708	57.9
YOLOv8s (본 시스템)	1280	0.438	0.670	25.5
YOLOv8m	640	0.567	0.759	34.2

[1] 모델이 클수록 mAP 는 오르고 속도는 떨어졌습니다. s 는 n 보다 mAP50-95 가 0.075 높으면서 속도는 비슷했습니다.

[2] COCO 에서는 1280 이 640 보다 낮았습니다. 모델이 640 으로 학습됐고 COCO 물체는 대부분 중간 크기 이상이기 때문입니다.

[3] 공식 mAP(80 개 클래스 기준, YOLOv8n 37.3 · s 44.9 · m 50.2)는 5 개 클래스 평균인

직접 측정값과 직접 비교하지 않습니다.



발표자료 8 장 — 탐지 모델 선택 (YOLOv8s · 1280 과 그 근거)

[입력 해상도 비교 — compare_resolution.py (2026.10.15)]

본 시스템이 1280 을 쓰는 근거는 테스트 영상의 작은 보행자입니다. road.mp4 에서 고르게 뽑은 60 프레임을 YOLOv8s(사람만, conf 0.25)로 탐지했습니다.

입력 크기	사람 탐지	작은 사람 (높이 40px 미만)
640	12	5
1280	190	100

일반적으로 1280 이 더 정확한 것이 아니라, 높은 곳에서 찍은 이 영상처럼 물체가 작을 때 유리하다는 점을 확인했습니다.

[처리 속도 (삼성역 영상 150 프레임, 탐지만 · FPS)]

모델	CPU 640	CPU 1280	GPU 640	GPU 1280
YOLOv8n	21.8	6.6	70.7	46.6
YOLOv8s (본 시스템)	9.6	2.6	63.4	27.8
YOLOv8m	4.2	1.1	37.0	14.4

추적 · 화면 표시 · 영상 저장까지 포함한 전체 처리(90 초)는 CPU 19 분(약 2.4 FPS), GPU 238 초(약 11 FPS)가 걸렸고, 탐지 · 카운팅 결과는 두 장치에서 같았습니다.

프로젝트 일지(4 차)

작업내역

<프로젝트 목표>

회차	목표	기간
4 차	발표 (발표자료 20 장, 대본·예상 질문)	26.10.13 ~ 26.10.16

<요약>

발표자료 20 장(TrafficLens_발표_20 장.pptx)을 만들어 2026.10.13 (화) 발표.

측정이 끝난 mAP 결과를 8 장에 반영하고, 장마다 발표자 노트(대본 · 동작 · 넘기는 말 · 키워드 · 시간) 작성.

발표 준비 자료(대본, 외울 숫자, 예상 질문 31 개)와 보조 모니터용 큰 글씨 대본 제작.

세부내역

[발표자료 · 발표]

발표자료(TrafficLens_발표_20 장.pptx)는 20 장, 약 10 분 15 초 분량이며 2026.10.13 (화)에 발표했습니다. 문제와 목표 → 시스템과 방법 → 개선 → 결과와 시연의 4 개 장으로 구성했고, 시연에는 중복 박스 제거를 반영한 result_improved_v2.mp4 를 썼습니다.

10.15 에 측정한 mAP 표를 8 장에 넣고, 1280 을 쓰는 근거를 '테스트 영상의 작은 보행자' 로 고쳐 썼습니다. 장마다 발표자 노트에 대본 · 동작 · 넘기는 말 · 키워드 · 시간을 적었습니다.



발표자료 1 장 (표지) — 장창근 · 2026.10.13

[발표 자료 목록]

자료	파일	내용

발표 슬라이드	TrafficLens_발표_20 장.pptx · .pdf	20 장, 장마다 발표자 노트
노트 포함 PDF	TrafficLens_slides_with_notes.pdf	한 쪽에 슬라이드 1 장 + 대본
발표 준비 자료	TrafficLens_presenter_kit.docx · .pdf	1 부 대본, 2 부 키워드 · 외울 숫자, 3 부 예상 질문과 답변 31 개
큰 글씨 대본	TrafficLens_script_20.pptx	보조 모니터용, 어두운 배경에 대본만 28 ~ 48pt

프로젝트 일지(5 차)

작업내역

<프로젝트 목표>

회차	목표	기간
5 차	소개 사이트·보고서·Colab 노트북·코드 저장소 정리	26.10.13 ~ 26.10.16

<요약>

소개 사이트 trafficlens.maxfinup.com 에 개요 · 보고서 · 발표 슬라이드 · 자료실을 올리고 측정 결과를 반영.

보고서 7 개 장 + 부록, 같은 실험을 재현하는 Colab 실습 노트북 작성.

코드와 문서를 GitHub 비공개 저장소로 관리(영상 · 가중치 파일 제외).

세부내역

[소개 사이트 — trafficlens.maxfinup.com]

Django 사이트(maxfinup.com)의 프로젝트 페이지를 trafficlens.maxfinup.com 서브도메인으로 연결하고 전용 헤더 · 로고 · 꼬리말을 붙였습니다.

주소	페이지	내용
/	프로젝트 개요	한눈에 보기 · 실행 영상 · 처리 흐름 · 결과 · 로고 시안 7 종(01 렌즈 도로 채택)
/report/	보고서	서론부터 실험 결과 · 결론, 부록(발표 자료)까지 보고서 전문
/slides/	발표 슬라이드	발표자료 20 장과 장별 발표 대본
/#tl-files	자료실	Colab 노트북, traffic_detect.py, evaluate.py, 개선 버전 미리보기 영상

[시스템 구조]

사용자 → Cloudflare (프록시 · TLS) → nginx (trafficlens.maxfinup.com) → gunicorn → Django (maxfinup.com 프로젝트 페이지)

- **서버** Oracle Cloud 서버, git pull → collectstatic → gunicorn 재시작 순서로 수동 배포
- **슬라이드** PPTX 에서 내보낸 슬라이드 JPG 20 장 + 장별 대본
- **자료실** Colab 노트북 · 실행 코드 · 측정 코드 · 미리보기 영상을 정적 파일로 제공

작업 이력: 10.14 발표 자료를 20 장 최신본(10.13 발표)으로 교체, 개요 페이지에 로고

시안 7 종 추가 → 10.15 8 장 mAP 표 · 보고서 6.2 절 직접 측정 mAP 반영 → 10.16 발표 대본 교체, 보고서 부록(발표 자료) 추가, Colab 노트북 갱신

[보고서 · Colab 노트북 · 코드 저장소]

- **보고서** 서론 → YOLO 개요 → 개발 환경 → 데이터셋과 탐지 클래스 → 시스템 설계 및 구현 → 실험 결과 및 분석 → 결론 및 향후 개선점의 7 개 장 + 부록 (TrafficLens_report.docx, 사이트 /report/)
- **Colab 노트북** TrafficLens_Colab.ipynb — 이론, 실험 4 개, 결과 그래프, 고찰, 제출 체크리스트. 2 차 개선 실험은 실험 2 · 3 으로 재현
- **코드 저장소** 2026.10.14 GitHub 비공개 저장소(changkeunjang1/trafficlens) 생성, 용량이 큰 영상(mp4) · 가중치(pt) 파일은 제외

사진 첨부)

- 발표 자료: 슬라이드 20 장과 장별 대본



- 보고서 전문


TrafficLens
YOLO 팀 로드비전 · 도로 위 차량·보행자 인식
개요
보고서
발표 자료
자료실
PROJECT · YOLO 과제
TrafficLens
YOLO 기반 도로 위 차량·보행자 실시간 인식 시스템
팀 로드비전 · 팀장 장창근 · 팀원 반광수, 지명준, 장혜원 · MBC 아카데미
개요
보고서
발표 자료
목차
1. 서론
2. YOLO 개요
3. 개발 환경
4. 데이터셋과 탐지 클래스
5. 시스템 설계 및 구현
6. 실험 결과 및 분석
6.5 개선 실험
7. 결론 및 향후 개선점
참고문헌
부록. 발표 자료
YOLOv8을 활용한 교통 영상 객체 탐지 및 성능 분석 · 팀 로드비전 · 2026년 10월 12일
1. 서론
본 과제는 TrafficLens 시스템을 통해 YOLOv8 모델로 도로 영상 속 차량과 보행자를 실시간으로 탐지하고, 종류별 개수를 화면에 표시하는 시스템을 구현한다.
1.1 연구 배경
도로 위 차량과 보행자를 사람이 직접 감시하는 방식은 인력과 시간이 많이 들고, 놓치는 상황이 생기기 쉽다. 딥러닝 기반 객체 탐지 기술은 CCTV나 차량 카메라 영상에서 객체를 자동으로 찾아내 이 문제를 줄일 수 있다. 특히 YOLO는 속도가 빨라 실시간 처리가 필요한 교통 분야에 적합하다.
1.2 연구 목적
• 사전 학습된 YOLOv8 모델로 차량(승용차, 버스, 트럭, 오토바이)과 보행자를 탐지한다.
• 탐지된 객체를 종류별로 세어 화면에 실시간으로 표시한다.
• 모델 크기별 정확도(mAP)와 처리 속도(FPS)를 비교해 실시간 적용 가능성을 확인한다.
1.3 활용 분야

프로젝트 일지(6 차)

작업내역

<프로젝트 목표>

회차	목표	기간
6 차	정답(직접 센 수) 비교 평가 및 최종 마무리	26.10.19 ~ 26.10.31

<요약>

사람이 직접 센 정답과 프로그램의 line 값을 비교하려고 1 분 클립 · 세기용 영상 · 기록표와 세는 규칙을 준비.

2026.10.26 현재 정답 기록표(count_sheet.csv)는 비어 있어 오차 결과는 측정 중.

오차표가 나오면 발표자료 15 · 16 · 20 장, 보고서, 사이트에 함께 반영하고 10.30 제출.

세부내역

[정답 구간 준비 — gt/]

사람이 직접 센 정답과 프로그램의 line 값을 비교하려고 road.mp4 에서 1 분 구간을 잘라 세기 자료를 만들었습니다.

파일	내용
gt_clip.mp4	road.mp4 의 0:30 ~ 1:30 (1 분, 1,797 프레임), 프로그램 실행용
gt_clip_count.mp4	같은 영상에 노란 기준선 · 방향 · 시간 · 프레임 번호를 넣은 세기용
count_sheet.csv	10 초 단위 기록표 (클래스 × 방향, 메모)



세기용 영상(gt_clip_count.mp4) — 노란 기준선, LEFT / RIGHT 방향, 오른쪽 아래 시간 · 프레임 번호

[세는 규칙 (프로그램과 같은 기준)]

[1] 차량 · 사람의 중심이 노란선을 넘는 순간 1 번 센다. 선 끝 밖으로 지나가면 세지 않는다.

[2] 같은 대상은 한 번만 센다 (선 위에서 멈췄다 다시 가도 1).

[3] 클래스: Car(승용 · SUV · 택시) / Bus / Truck(화물 · 승합 트럭) / Motorcycle / Person.

[4] 10 초마다 멈추고 기록, 0.5 배속 권장. 두 사람이 따로 세어 다르면 다시 확인한다.

[진행 현황 (2026.10.26 기준)]

정답 기록표(count_sheet.csv)는 아직 비어 있어 정답 대비 오차는 측정 중입니다. 남은 작업은 아래와 같습니다.

- 정답 세기 — 두 사람이 따로 세어 count_sheet.csv 기록
- gt_clip.mp4 로 기본(--baseline) · 개선 버전 실행, 클래스별 line 값과 정답 비교
- 오차표를 발표자료 15 · 16 · 20 장, 보고서 6 장, 사이트에 반영
- 최종 제출 (마감 2026.10.30)

[프로젝트 전체 정리 (2026.10.28 기준)]

시스템 · 측정 · 문서 · 사이트는 모두 준비되었고, 남은 핵심은 사람이 센 정답과 비교한 오차 숫자 하나입니다.

기간이 3 주라 직접 학습할 데이터와 시간이 없어, 학습 없이 COCO 사전학습 YOLOv8s 에 ByteTrack 추적 · 개선 필터 · 기준선 카운팅을 더하는 방식을 택했습니다. 모델 학습보다 사전학습 모델을 현장에 맞게 쓰는 기술에 집중했습니다.

진행 흐름

- **10.12** 기본 · 개선 버전 코드, evaluate.py, 로고 7 종 — 개선 필터 3 개 (정지 물체, 클래스 다수결, 짧은 추적 제거)
- **10.13** 1 차 발표, 오탐 원인 분석 — 주차 트럭 위 Bus 중복 박스 → 중복 박스 제거, 버스 total 67 → 29
- **10.14** GitHub 저장소, 발표자료 PPTX 정리 — 코드 · 문서만 저장소에 (영상 · 가중치 제외)
- **10.15** mAP · FPS 직접 측정, 해상도 비교 — 1280 근거: 작은 보행자 탐지 12 → 190 명
- **10.16** 발표 준비 자료 — 대본, 예상 질문 31 개, 큰 글씨 대본, 노트 포함 PDF
- **10.26** 작업일지, 보고서 PDF, 사이트 자료실 — 사이트에서 자료 내려받기

핵심 결과

- 기준선 통과 차량 151 대는 기본 · 개선 버전이 같고, 차량 total 753 → 485 (−36%),

보행자 total 75 → 35: 세야 할 것은 그대로, 노이즈만 줄임

- 1280 입력은 COCO 정확도가 더 높아서가 아니라, 이 영상의 먼 보행자를 잡기 위해 선택 (mAP 표는 3 차 일지)
- 처리 속도는 GTX 1060 에서 탐지만 약 26 ~ 28 FPS, 추적 · 저장 포함 약 12 FPS

마무리 방침 (10.28 ~ 10.30)

[1] 정답 세기를 가장 먼저: 1 분 클립을 10 초 구간별로 세어 오차율 표 하나를 만든다. 지금 가장 약한 부분이자 효과가 가장 큰 일이다.

[2] 오차 숫자가 나오면 발표자료 15 · 16 · 20 장, 예상 질문 Q19, 보고서 6 장, 사이트, 작업일지 6 차, 현황 페이지에 한 번에 반영한다.

[3] 팀원 역할과 자체 평가 의견은 팀이 직접 작성한다.

[4] 새 기능은 추가하지 않는다. 기능을 바꾸면 모든 숫자를 다시 재야 하므로 측정과 정리로 마무리한다.

[5] 발표 연습: 대본 10 분 15 초, 큰 글씨 대본으로 1 ~ 2 회. 자료 요청은 사이트 자료실, 진행 과정 질문은 작업일지로 안내한다.

[활용 방안 및 기대 효과]

- **현장 활용** 스마트 교차로의 방향별 교통량 조사 · 신호 제어, 어린이 보호구역 보행자 감지 · 경고
- **기대 효과** 학습 없이 사전학습 모델로 즉시 적용, 기준선만 바꾸면 다른 교차로 · 웹캠에도 사용 (--line, --source)
- **확장성** AI Hub 한국 도로 데이터로 fine-tuning 해 국내 버스 · 트럭 인식률 향상, 자율주행 주변 인식으로 확장

[결론 및 향후 과제]

성과

- YOLOv8 + ByteTrack 으로 학습 없이 도로 영상의 차량 · 보행자를 탐지하고 기준선 통과 교통량을 종류 · 방향별로 집계 (90 초 차량 151 대)
- 개선 기능 4 종으로 차량 total 36%, 보행자 total 53% 감소, 중복 박스 제거로 버스 total 67 → 29
- 모델 · 해상도별 mAP · FPS 를 직접 측정하고, 1280 입력의 근거(작은 보행자 12 → 190 명)를 확인
- 소개 사이트 · 보고서 · 발표자료 · Colab 노트북으로 재현 가능하게 정리

한계

- 전체 처리 속도는 GPU 약 11 FPS(영상 저장 포함)로 실시간(30 FPS)에 못 미침
- 정지 물체 필터는 신호등 5 개 중 3 개만 걸렀고, 신호를 기다리는 보행자도 빠질 수

있음

- 횡단보도 위 작은 보행자는 여전히 자주 놓침, 정답 대비 오차는 측정 중

향후 과제

- 영상 저장 해상도를 낮추거나 별도 스레드에서 저장해 전체 처리도 실시간에 가깝게
- 화면을 여러 조각으로 나눠 탐지(타일링)해 작은 보행자 검출력 향상
- 높은 시점의 보행자 · 한국 도로 데이터로 추가 학습

한 줄 요약: 누적 개수(total)보다 기준선 통과 수(line)가 실제 교통량에 가깝다 — 무엇을 세는지 정하는 것이 성능만큼 중요하다.

[자체 평가 의견]

팀원	의견
장창근 (팀장)	
반광수	
지명준	
장혜원	